RL78 ファミリ
ボードサポートパッケージモジュール
Firmware Integration Technology

要旨

Firmware Integration Technology（以下 FIT と称す）モジュールを使用するプロジェクトの基盤となるのがルネサスボードサポートパッケージ FIT モジュール（r_bsp）です。r_bsp は設定が簡単で、リセットから main()関数までに MCU と使用するボードが必要とする全てのコードを提供します。本ドキュメントでは、r_bsp の規約を説明し、その使用方法、設定方法、ご使用のボードに対応した BSP の作成方法を紹介します。

動作確認デバイス

- RL78/G23 グループ

本アプリケーションノートを他のマイコンへ適用する場合、そのマイコンの仕様にあわせて変更し、十分評価してください。

対象コンパイラ

- Renesas Electronics C/C++ Compiler Package for RL78 Family
- IAR C/C++ Compiler for Renesas RL78
- LLVM C/C++ Compiler for Renesas RL78

各コンパイラの動作確認内容については 7.1 動作確認環境を参照してください。

一部機能には制限があります。4.4 制限事項を参照してください。

目次

1. 概要 3
2. 機能 6
3. コンフィギュレーション 11
4. API 情報 18
5. API 関数 22
6. プロジェクトのセットアップ 27
7. 付録 32
改訂記録 32

1. 概要

MCU を正しく設定するためには、ユーザアプリケーションを実行する前に、一連の作業を行う必要があります。必要な作業や量は使用する MCU によって異なります。一般的な例としては、スタックの設定、メモリの初期化、CPU/周辺ハードウェア・クロックの設定、ポートの端子の設定などがあります。これらの設定は本書で示す手順に沿って行う必要があります。r_bsp は、これらの設定を簡単に行えるように提供されるものです。

r_bsp は、ご使用の MCU がリセットからユーザアプリケーションの main()関数を開始するまでに必要な要素を提供します。また、r_bsp には、多くのアプリケーションで必要となる共通の機能も備えられています。それらの中には、クロックの発振/停止を設定するための関数や CPU/周辺ハードウェア・クロックの周波数を取得する関数などがあります。

すべてのアプリケーションでリセット後に必要な手順は同じですが、各設定の内容も同じというわけではありません。例えば、アプリケーションごとに、スタックサイズや使用するクロックが異なります。r_bsp の設定に関するオプションはコンフィグヘッダファイルに納められているので、簡単に設定オプションを変更することができます。

1.1 用語

用語	説明
プラットフォーム	ユーザの開発ボード。「ボード」を使用する場合もあり。
BSP	ボードサポートパッケージの略称。

1.2　ファイル構成

r_bsp のファイル構成を図 1.1 に示します。r_bsp フォルダの下に、3 つのフォルダと 2 つのファイルがあります。

doc フォルダには r_bsp のドキュメントが含まれます。

board フォルダには、generic フォルダがあります。

generic フォルダには、MCU ごとにフォルダがあります。

generic フォルダの構成を図 1.2 に示します。

mcu フォルダには、MCU ごとに 1 フォルダが含まれます。mcu フォルダには他に all フォルダがあり、このフォルダには r_bsp で全 MCU に共通のソースが含まれます。

platform.h は、ユーザが開発プラットフォームを選択するためのファイルで、ユーザプロジェクトに必要なすべてのヘッダファイルを board および mcu フォルダから選択します。これについては、後のセクションで詳しく説明します。

readme.txt には r_bsp に関する情報が要約されています。

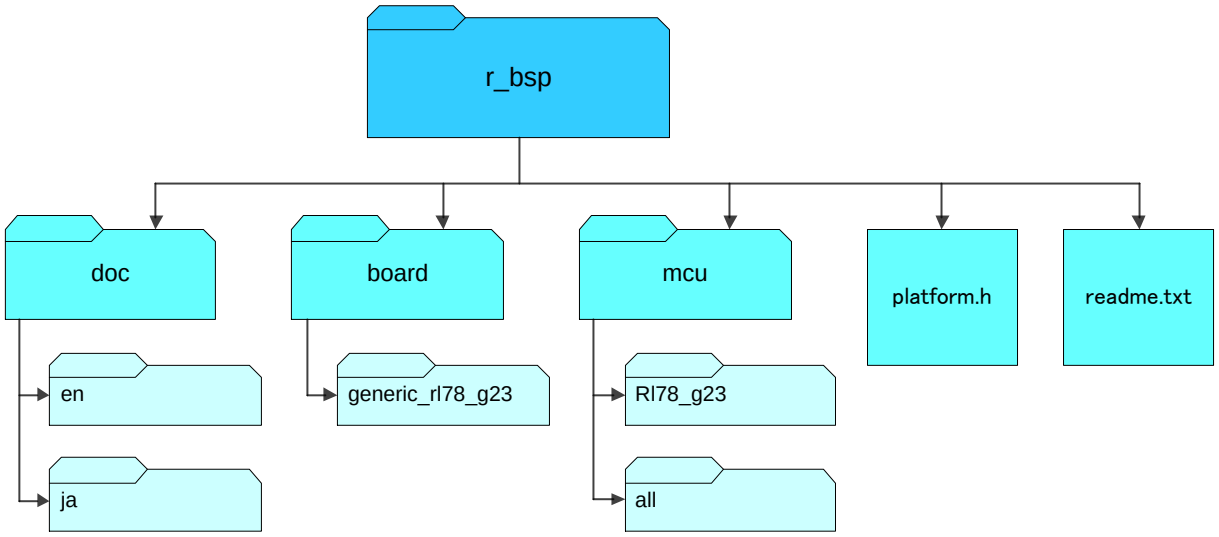


図 1.1　r_bsp ファイル構成

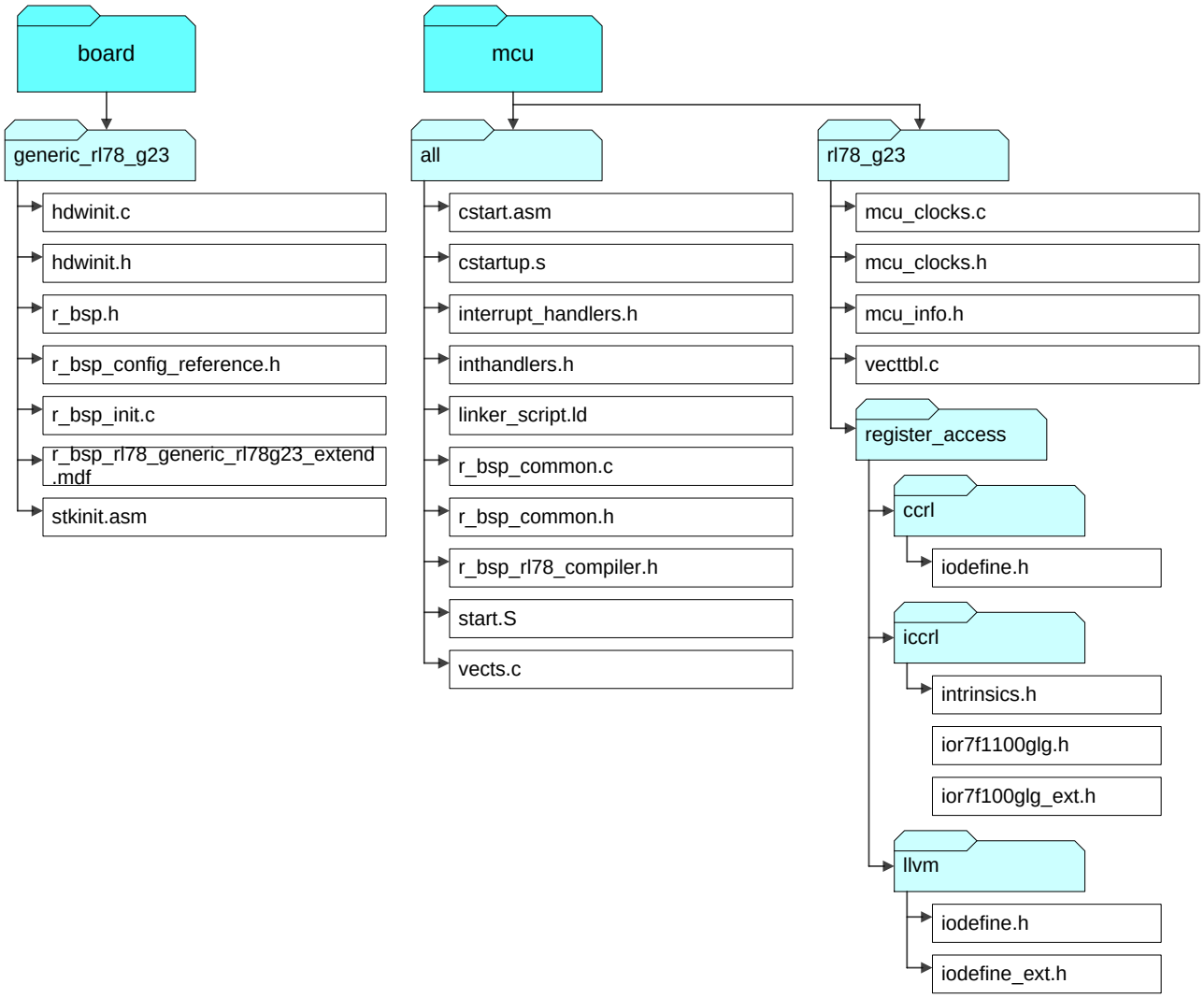


図 1.2　generic フォルダの構成

2. 機能

ここでは、r_bsp 搭載の機能について詳しく説明していきます。

2.1 MCU 情報

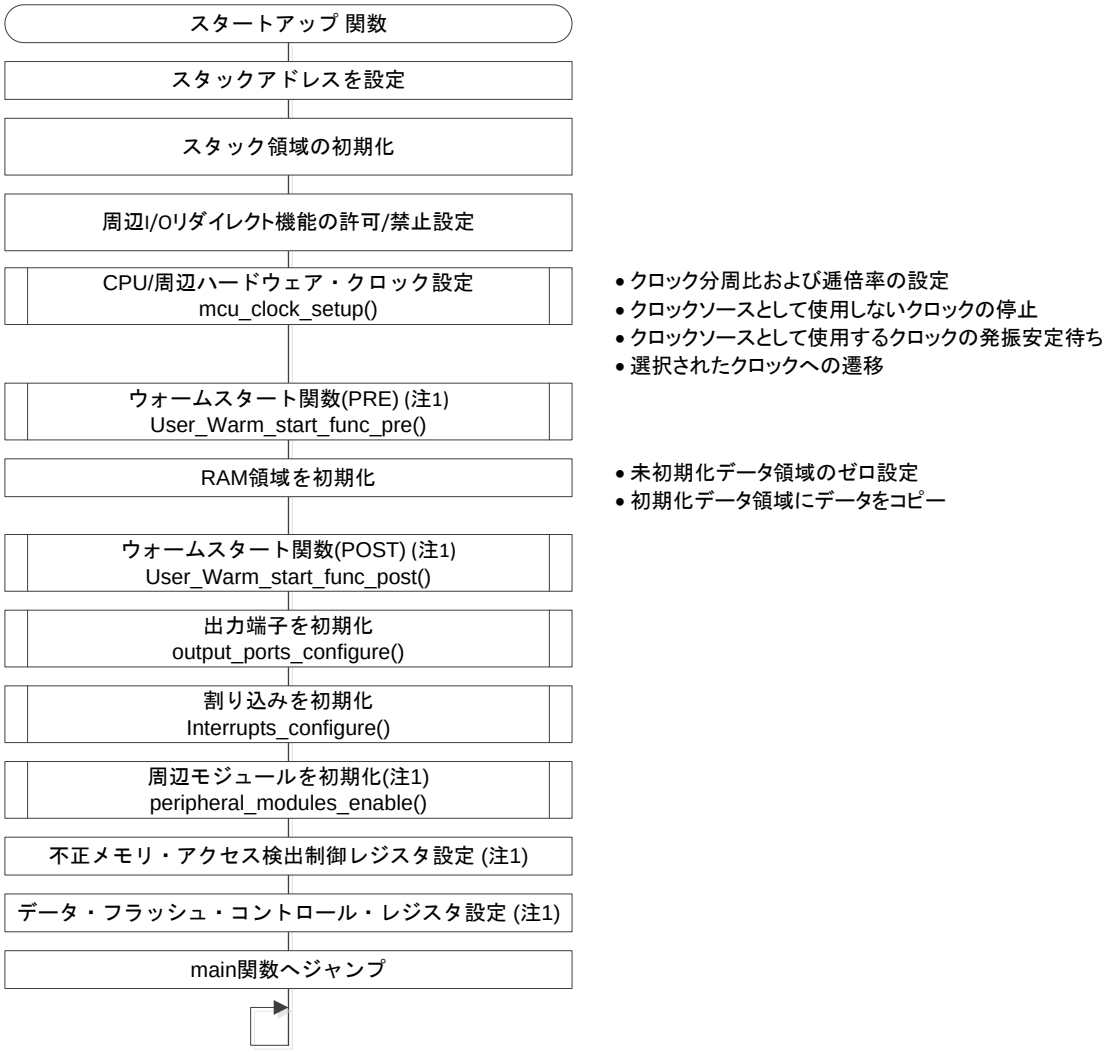
r_bsp の強みは、システム全体の設定をプロジェクトの 1 カ所で 1 度だけ定義すれば、その設定を共通で使用できるという点です。この情報は r_bsp で定義され、FIT モジュール、およびユーザコードで使用できます。FIT モジュールはこの情報を使って、自動的にコードをユーザのシステムに応じて設定します。r_bsp でこの情報が提供されなければ、ユーザが FIT モジュールごとに個別にシステム情報を設定する必要があります。

r_bsp の設定については、3 章で説明します。r_bsp はこの設定情報を使って、mcu_info.h のマクロ定義を設定します。MCU の mcu_info.h に存在するマクロの例を以下に示します。

定義	説明
BSP_<CLOCK>_HZ	MCU 上の各クロックについてこれらのマクロの 1 つが対応します。各マクロは、そのクロックの周波数を（ヘルツ単位で）定義します。たとえば、BSP_LOCO_HZ は、LOCO の周波数（Hz）を定義します。 BSP_SUB_CLOCK_HZ は、サブシステム・クロック（Hz）を定義します。

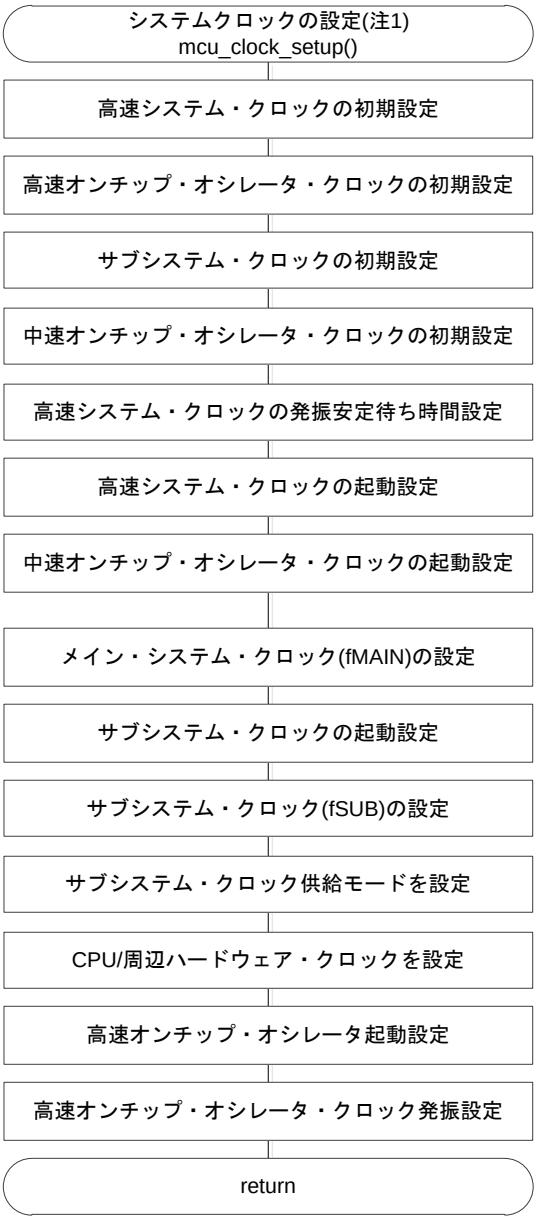
2.2　初期設定

Renesas コンパイラを使用する場合は_start 関数を、LLVM を使用する場合は PowerON_Reset 関数を MCU のリセットベクタとして設定します。IAR コンパイラを使用する場合は__iar_program_start 関数を MCU のリセットベクタとして設定します。_start と PowerON_Reset_PC と__iar_program_start 関数(スタートアップ関数)は、MCU がユーザアプリケーションを使用できる状態にするために、様々な初期化処理を行います。以下に、スタートアップ関数と CPU/周辺ハードウェア・クロック設定の動作をフローチャートで示します。



注1: r_bsp_config.hの設定によって、動作は異なります。

図 2.1　スタートアップ関数のフローチャート



注1: r_bsp_config.hの設定によって、動作は異なります。

図 2.2　CPU/周辺ハードウェア・クロック設定のフローチャート

2.3　グローバル割り込み

リセット解除後、割り込みは禁止になっています。必要に応じて割り込みを許可してください。

割り込みを許可する場合は”BSP_EI”関数を、禁止する場合は”BSP_DI”関数を使用してください。

詳細は 5.1 概要を参照してください。

RL78 のデバイスには固定ベクタテーブルがあります。固定ベクタテーブルはメモリマップ先頭の固定のロケーションに配置されます。

固定ベクタテーブルは Renesas コンパイラまたは LLVM を使用する場合は iodefine.h に、IAR を使用する場合は iorxxx.h(注 1) に定義されます。

(注 1) xxx はデバイスによって異なります。

2.4　クロックの設定

CPU/周辺ハードウェア・クロックは r_bsp の初期化処理で設定されます。クロックは、r_bsp_config.h ファイルのユーザ設定に従って設定されます（3.2.5 参照）。クロックの設定は、C ランタイム環境の初期化処理の前に行われます。クロック選択時、r_bsp のコードによって、選択されたクロックが安定するのに要する遅延時間が取られます。

2.5　スタック領域

スタックはリセット後にスタートアップ関数内で設定、および初期化されます。

IAR コンパイラを使用する場合は GUI でスタックサイズを設定してください。

2.6　ID コード

RL78 MCU には ROM にある 10 バイトの ID コードを使って、デバッガを介しての MCU メモリの読み出し、シリアルブートモードでの MCU メモリの読み出し、あるいはデバイスからのファームウェアの取り出しが行われないように保護します。ID コードは、オンチップ・デバッグ・セキュリティ ID 設定メモリに配置されます。セキュリティ ID の値は Renesas コンパイラ環境ではコンパイルオプションで設定します。IAR 環境または LLVM 環境では r_bsp_config.h で設定します。ID コードのオプションに関する詳細は、ユーザーズマニュアル ハードウェア編の「オプション・バイト」章や「オンチップ・デバッグ機能」章を参照ください。

2.7　オプション・バイト

RL78 MCU はフラッシュ・メモリにオプション・バイトが配置されています。電源投入時またはリセットからの起動時に、自動的にオプション・バイトを参照して、指定された機能の設定を行います。オプション・バイトにはウォッチドッグ・タイマの設定や電圧検出回路の設定などがあります。オプション・バイトに設定する値は、Renesas コンパイラ環境ではコンパイルオプションで設定します。IAR 環境または LLVM 環境では r_bsp_config.h で設定します（3.2.6 参照）。

2.8　RAM/SFR ガード機能

RL78 MCU には不正メモリ・アクセス検出制御レジスタがあり、指定した RAM 空間のデータおよびポート機能、割り込み機能、クロック制御機能、電圧検出回路、RAM パリティ・エラー検出機能の制御レジスタのデータを保護します。設定に使用する値は r_bsp_config.h で設定できます。

2.9　CPU 機能

割り込みの許可/禁止など、CPU 機能に関する設定を行う API 関数が用意されています。詳細は 5 章をご覧ください。

2.10　スタートアップ無効化

スタートアップを無効化する場合、スタートアップアセンブラコードを手動で削除してください。

環境とスタートアップアセンブラの関係は以下の通りです。

・Renesas コンパイラ　：cstart.asm

・LLVM コンパイラ　：start.S

・IAR コンパイラ　：cstartup.s

また、ユーザスタートアップを追加してください。

2.10.1　スタートアップ無効化の設定方法

BSP のスタートアップ処理を無効化する場合は、以下の設定をしてください。

(1)コンフィグレーションファイルの設定

ユーザが作成したスタートアップ処理の内容を r_bsp_config.h に設定してください。BSP の API 関数や周辺の FIT モジュールは r_bsp_config.h の内容を参照する場合があります。ユーザが作成したスタートアップ処理の内容と r_bsp_config.h の内容に差異がある場合、FIT モジュールは正常に動作しない場合があります。

各周辺 FIT モジュールから参照される BSP の情報は、r_bsp_config.h から生成されるため、ユーザが作成したスタートアップ処理の内容と r_bsp_config.h に設定された内容を同じにする必要があります。

図 2.3 にコンフィグレーションファイルの設定を示します。

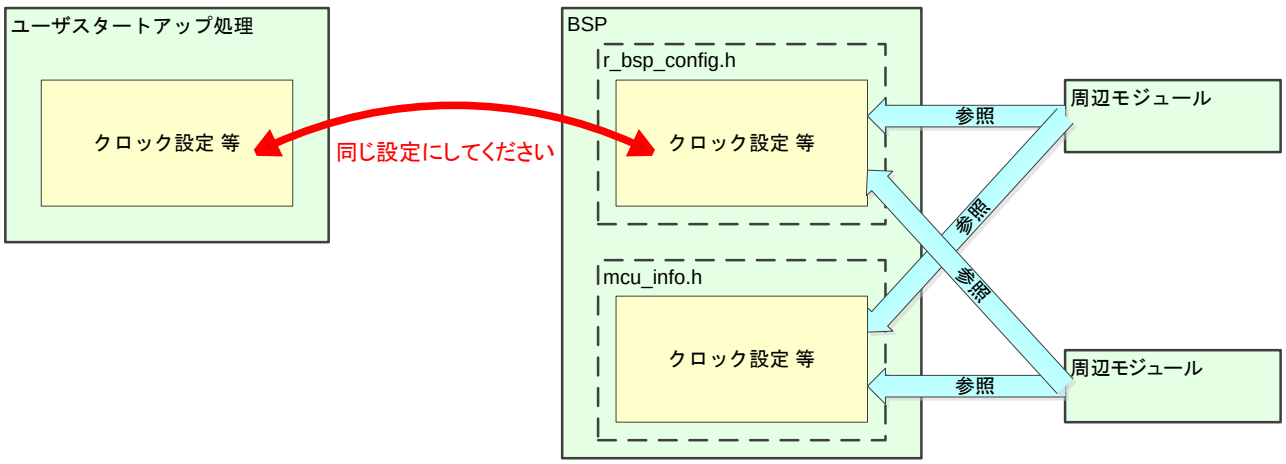


図 2.3　コンフィグレーションファイルの設定

3.　コンフィギュレーション

r_bsp では、2 つのヘッダファイルを使って設定を行います。1 つはプラットフォームの選択を、他方は選択したプラットフォームの設定を行います。

3.1　プラットフォームを選択する

r_bsp は様々な MCU に対応するボードサポートパッケージを提供します。r_bsp フォルダの直下にある platform.h を変更して、使用するプラットフォームを選択します。

バージョン 1.00 では RL78/G23 にのみ対応しているため、プラットフォームの選択は不要です。

3.2　プラットフォームの設定

プラットフォームを選択したら、次にそのプラットフォームの設定を行わなければなりません。プラットフォームの設定は r_bsp_config.h を使って行います。プラットフォームごとに、r_bsp_config_reference.h というコンフィギュレーションファイルがあり、各プラットフォームの board フォルダに格納されています。

r_bsp_config.h の内容はそれぞれの MCU によって異なりますが、同じオプションも多数あります。以降のセクションでは、それらの設定オプションについて詳しく説明します。各マクロは共通で ‘BSP_CFG_’から始まっており、検索や識別が簡単に行えます。

Smart Configurator を使用する場合は、ソフトウェアコンポーネント設定画面でコンフィグレーションオプションを設定できます。設定値はモジュールを追加する際に、自動的に r_bsp_config.h に反映されます。

3.2.1　MCU 製品型名情報

MCU の製品型名情報によって、MCU の様々な情報とともに r_bsp を提供できます。コンフィギュレーションファイルの先頭には、MCU の製品型名に関する情報が定義されます。これらのマクロ名はすべて‘BSP_CFG_MCU_PART’から始まります。MCU によって製品型名に関する情報量は異なりますが、以下に標準的な定義を示します。

表 3.1　製品型名の定義

定義	値	説明
BSP_CFG_MCU_PART_ROM_SIZE	r_bsp_config.h にて、#define の上にあるコメントを参照。	ROM のサイズを定義します。
BSP_CFG_MCU_PART_PIN_NUM		ピン数を定義します。
BSP_CFG_MCU_PART_HAS_DATA_FLASH		フラッシュ・メモリ搭載の有無を定義します。
BSP_CFG_MCU_PART_ROM_TYPE		デバイスの種類を定義します。

3.2.2　周辺 I/O リダイレクション・レジスタ

RL78 MCU には、兼用機能を割り当てるポートを切り替える機能があります。リセット後、r_bsp は、r_bsp_config.h の端子割り当て設定マクロを使って、MCU の端子割り当てを設定します。

表 3.2　周辺 I/O リダイレクション・レジスタの定義

定義	値	説明
BSP_CFG_PIOR0	r_bsp_config.h にて、#define の上にあるコメントを参照。	兼用機能を割り当てるポートを定義します。 TI02/TO02, TI03/TO03, TI04/TO04, TI05/TO05, TI06/TO06, TI07/TO07
BSP_CFG_PIOR1		兼用機能を割り当てるポートを定義します。 INTP10, INTP11, TxD2, RxD2, SCL20, SDA20, SI20, SO20, SCK20, TxD0, RxD0, SCL00, SDA00, SI00, SO00, SCK00
BSP_CFG_PIOR2		兼用機能を割り当てるポートを定義します。 SCLA0, SDAA0
BSP_CFG_PIOR3		兼用機能を割り当てるポートを定義します。 PCLBUZ0
BSP_CFG_PIOR4		兼用機能を割り当てるポートを定義します。 PCLBUZ1, INTP5
BSP_CFG_PIOR5		兼用機能を割り当てるポートを定義します。 INTP1, INTP3, INTP4, INTP6, INTP7, INTP8, INTP9, TxD1, RxD1, SCL10, SDA10, SI10, SO10, SCK10

3.2.3　RAM/SFR ガード機能

RL78 MCU には指定した RAM 空間のデータおよびポート機能、割り込み機能、クロック制御機能、電圧検出回路、RAM パリティ・エラー検出機能の制御レジスタのデータを保護する機能があります。リセット後、r_bsp は、r_bsp_config.h のガード機能設定マクロを使って、MCU のガード領域を設定します。

表 3.3　RAM/SFR ガード機能の定義

定義	値	説明
BSP_CFG_INVALID_MEMORY_ACCESS_DETECTION_ENABLE	r_bsp_config.h にて、#define の上にあるコメントを参照。	不正メモリ・アクセスの検出をするかしないかを定義します。
BSP_CFG_RAM_GUARD_FUNC		RAM ガード空間のサイズを定義します。
BSP_CFG_PORT_FUNCTION_GUARD		ポート機能の制御レジスタをガードするかしないかを定義します。
BSP_CFG_INT_FUNCTION_GUARD		割り込み機能のレジスタをガードするかしないかを定義します。
BSP_CFG_CHIP_STATE_CTRL_GUARD		クロック制御機能、電圧検出回路、RAM パリティ・エラー検出機能の制御レジスタをガードするかしないかを定義します。

3.2.4　データ・フラッシュのアクセス制限

RL78 MCU にはデータ・フラッシュへのアクセスを許可/禁止する機能があります。リセット後、r_bsp は、r_bsp_config.h のデータ・フラッシュのアクセス制限機能設定マクロを使って、データ・フラッシュへのアクセスを設定します。

表 3.4　データ・フラッシュのアクセス制限の定義

定義	値	説明
BSP_CFG_DATA_FLASH_ACCESS_ENABLE	r_bsp_config.h にて、#define の上にあるコメントを参照。	データ・フラッシュへのアクセスを許可するか禁止するかを定義します。

3.2.5　クロックの設定

RL78 MCU が使用できるクロックは MCU によって異なりますが、基本的な概念はすべてに共通です。リセット後、r_bsp は、r_bsp_config.h のクロック設定マクロを使って、MCU クロックを初期化します。

表 3.5　クロック設定の定義 (1/2)

定義	値	説明
BSP_CFG_HISYSCLK_SOURCE	0=ポート 1=水晶/セラミック発振子接続 2=外部クロック入力	高速システム・クロックの発振源を定義します。
BSP_CFG_SUBCLK_SOURCE	0=入力ポート 1=水晶発振子接続 2=外部クロック入力	サブシステム・クロックの発振源を定義します。
BSP_CFG_MOCO_SOURCE	0=中速オンチップ・オシレータ停止 1=中速オンチップ・オシレータ動作	中速オンチップ･オシレータ･クロックを動作する/停止するを定義します。
BSP_CFG_OCOCLK_SOURCE	0=高速オンチップ・オシレータ・クロック 1=中速オンチップ・オシレータ・クロック	メイン・オンチップ・オシレータ・クロック(f_{OCO})に使用するクロックソースを定義します。
BSP_CFG_MAINCLK_SOURCE	0=メイン・オンチップ・オシレータ・クロック(f_{OCO}) 1=高速システム・クロック(f_{MX})	メイン・システム・クロック(f_{MAIN})に使用するクロックソースを定義します。
BSP_CFG_SUBSYSCLK_SOURCE	0=サブ・クロック 1=低速オンチップ・オシレータ・クロック	サブシステム・クロックに使用するクロックソースを定義します。
BSP_CFG_FCLK_SOURCE	0=メイン・システム・クロック(f_{MAIN}) 1=サブシステム・クロック(f_{SUB})	CPU/周辺ハードウェア・クロック(f_{CLK})に使用するクロックソースを定義します。
BSP_CFG_XT1_OSCMODE	0=低消費発振 1(デフォルト) 1=通常発振 2=低消費発振 2 3=低消費発振 3	XT1 発振回路の発振モードを定義します。
BSP_CFG_FMX_HZ	高速システム･クロック周波数(単位: Hz)	高速システム・クロックの周波数を定義します。
BSP_CFG_X1_WAIT_TIME_SEL	0=$2^8/f_X$ 1=$2^9/f_X$ 2=$2^{10}/f_X$ 3=$2^{11}/f_X$ 4=$2^{13}/f_X$ 5=$2^{15}/f_X$ 6=$2^{17}/f_X$ 7=$2^{18}/f_X$	X1 クロックの発振安定時間を定義します。
BSP_CFG_ALLOW_FSUB_IN_STOPHALT	0=周辺機能へのサブシステム・クロック供給許可 1=リアルタイム・クロック以外の周辺機能へのサブシステム・クロック供給停止	STOP モード時およびサブシステム・クロックで CPU 動作中の HALT モード時のサブシステム・クロックの供給を定義します。

表 3.6　クロック設定の定義 (2/2)

定義	値	説明
BSP_CFG_RTC_OUT_CLK_SOURCE	0=サブシステム・クロック 1=低速オンチップ・オシレータ・クロック	リアルタイム・クロック、32 ビット・インターバル・タイマ、シリアル・インタフェース UART0,1、リモコン信号受信機能、クロック出力/ブザー出力制御回路の動作クロックを定義します。
BSP_CFG_HOCO_DIVIDE	(FRQSEL3=0 の場合) 0=f_{IH} : 24MHz 1=f_{IH} : 12MHz 2=f_{IH} : 6MHz 3=f_{IH} : 3MHz (FRQSEL3=1 の場合) 0=f_{IH} : 32MHz 1=f_{IH} : 16MHz 2=f_{IH} : 8MHz 3=f_{IH} : 4MHz 4=f_{IH} : 2MHz 5=f_{IH} : 1MHz	高速オンチップ・オシレータの周波数を定義します。 FRQSEL3 の設定はオプション・バイト(000C2H)を使用してください。 設定方法は 2.7 参照
BSP_CFG_WAKEUP_MODE	0=通常起動 1=高速起動	STOP モード解除および SNOOZE モード遷移時の高速オンチップ・オシレータ起動設定を定義します。
BSP_CFG_MOSC_DIVIDE	0=f_{MX} 1=f_{MX}/2 2=f_{MX}/4 3=f_{MX}/8 4=f_{MX}/16	高速システム・クロックの分周比を定義します。
BSP_CFG_MOCO_DIVIDE	0=4MHz 1=2MHz 2=1MHz	中速オンチップ・オシレータの周波数を定義します。
BSP_SUBWAITTIME	ループ回数(単位: 回)	サブシステム・クロック発振安定待ち時間を定義します。 メイン・システム・クロックを使用したループ回数を定義します。(注)
BSP _FIHWAITTIME	ループ回数(単位: 回)	高速オンチップ・オシレータ・クロックの発振安定待ち時間を定義します。 メイン・システム・クロックを使用したループ回数を定義します。(注)
BSP_FIMWAITTIME	ループ回数(単位: 回)	中速オンチップ・オシレータ・クロックの発振安定待ち時間を定義します。 メイン・システム・クロックを使用したループ回数を定義します。(注)
BSP_FILWAITTIME	ループ回数(単位: 回)	低速オンチップ・オシレータ・クロックの発振安定待ち時間を定義します。 メイン・システム・クロックを使用したループ回数を定義します。(注)
BSP_CFG_FIH_START_ON_STARTUP	0=高速オンチップ・オシレータ・クロックを停止する 1=高速オンチップ・オシレータ・クロックを起動する	初期化時の高速オンチップ・オシレータ・クロックの動作を定義する。

（注）ループ回数とは、nop 命令を 1 回実行する for 文によるループです。

実際のソースコードは以下の通りです。

```
/* WAIT_LOOP */
for (w_count = 0U; w_count <= BSP_SUBWAITTIME; w_count++)
{
    BSP_NOP();
}
```

ただし、実際のサイクル数は最適化オプションなどにより異なるため、ご使用の環境に合わせて設定してください。

3.2.6　オプション・バイト

オプション・バイトを設定して、リセット時の動作を選択できます。例えばウォッチドッグ・タイマや電圧検出回路の設定が行えます。

IAR 環境を使用する場合は r_bsp_config.h にオプション・バイトの設定値を定義してください。それ以外の環境ではプロジェクトのプロパティから設定してください。

表 3.6　オプション・バイトの定義

定義	値	説明
OPTBYTE0_VALUE OPTBYTE1_VALUE OPTBYTE2_VALUE OPTBYTE3_VALUE	オプション・バイトの値	オプション・バイトに設定する値を定義します。 本マクロ定義は IAR 環境でのみ使用されます。 Renesas コンパイラまたは LLVM 環境ではコンパイルオプションで設定します。

3.2.7　スタートアップ無効化

表 3.7　スタートアップ無効化の定義

定義	値	説明
BSP_CFG_STARTUP_DISABLE	0 = BSP スタートアップ有効 1 = BSP スタートアップ無効	クロックの初期設定処理の有効/無効を定義します。無効を選択し場合、クロックの初期設定処理が無効になります。 スタートアップ全体を無効化する場合は、スタートアップアセンブラを手動で削除し、ユーザスタートアップを追加してください。

3.2.8　Smart Configurator の使用

表 3.8　Smart Configurator の使用の定義

定義	値	説明
BSP_CFG_CONFIGURATOR_SELECT	0=Smart Configurator を使用しない 1=Smart Configurator を使用する	Smart Configurator を現在のプロジェクトで使用するかどうかを定義します。 BSP_CFG_CONFIGURATOR_SELECT = 1 の場合は、Smart Configurator 初期化関数が呼び出されます。

3.2.9　ウォームスタート時のコールバック機能

表 3.9　ウォームスタート時のコールバック機能の定義

定義	値	説明
BSP_CFG_USER_WARM_START_CALLBACK_PRE_INITC_ENABLED	0＝C ランタイム環境を初期化する前にユーザ関数を呼び出さない 1＝C ランタイム環境を初期化する前にユーザ関数を呼び出す	C ランタイム環境を初期化する前にユーザ関数を呼び出すかどうかを定義します。
BSP_CFG_USER_WARM_START_PRE_C_FUNCTION	C ランタイム環境が初期化される前に呼び出される関数	C ランタイム環境を初期化する前にユーザ関数を呼び出す際に呼び出される関数を定義します。
BSP_CFG_USER_WARM_START_CALLBACK_POST_INITC_ENABLED	0＝C ランタイム環境を初期化した後にユーザ関数を呼び出さない 1＝C ランタイム環境を初期化した後にユーザ関数を呼び出す	C ランタイム環境を初期化した後にユーザ関数を呼び出すかどうかを定義します。
BSP_CFG_USER_WARM_START_POST_C_FUNCTION	C ランタイム環境が初期化された後に呼び出される関数	C ランタイム環境を初期化した後にユーザ関数を呼び出す際に呼び出される関数を定義します。

4. API 情報

本ドライバ API はルネサスの API ネーミング規則に準じています。

4.1 ハードウェアの必要条件

適用外

4.2 ハードウェアリソースの必要条件

適用外

4.3 ソフトウェアの必要条件

なし

4.4 制限事項

4.4.1 IAR コンパイラに関する制限事項

IAR コンパイラではオプション・バイトの設定を r_bsp_config.h を使用して行ってください。

4.5 サポートされているツールチェーン

本 FIT モジュールは「7.1 動作確認環境」に示すツールチェーンで動作確認を行っています。

4.6 使用する割り込みベクタ

本 FIT モジュールで使用する割り込みベクタはありません。

4.7 ヘッダファイル

platform.h を組み込むことによって、すべての API 呼び出しがインクルードされます。platform.h は本ドライバのプロジェクトコードと一緒に提供されます。

4.8 整数型

本プロジェクトは、コードをわかりやすく、移植可能なものとするために、ANSI C99 “Exact width integer types” を使用しています。これらの型は stdint.h に定義されます。

4.9 API Typedef

4.9.1 クロックリソース

この typedef は、R_BSP_StartClock()、R_BSP_StopClock()、R_BSP_SetClockSource()関数で使用できるコマンドを定義します。

以下に、RL78/G23 を使用した場合の typedef を示します。

```
/* clock mode */
typedef enum
{
    HIOCLK,      // High-speed on-chip oscillator
    SYSCLK,      // High-speed system clock
    SXCLK,       // Subsystem clock
    MIOCLK,      // Middle-speed on-chip oscillator
    LOCLK        // Low-speed on-chip oscillator
} clock_mode_t;
```

4.10　戻り値

4.10.1　エラーコード

この typedef は、R_BSP_StartClock()、R_BSP_StopClock()、R_BSP_SetClockSource()関数から返すエラーコードを定義します。

以下に、RL78/G23 を使用した場合の typedef を示します。

```
/* Error identification */
typedef enum
{
    /* メンバは以下の表を参照ください。 */
} bsp_err_t;
```

メンバ	説明
BSP_OK	成功
BSP_ARG_ERROR	無効な引数が入力されました。
BSP_ERROR1	指定されたクロックが発振していません。
BSP_ERROR2	クロックのリソース切り替え中に、発振していないクロックリソースに切り替えられる可能性があります。
BSP_ERROR3	設定できない状態移行が指定されました。 ユーザーズマニュアルを参照してください。

4.11　コードサイズ

下表の値は下記条件で確認しています。本モジュールの ROM サイズ、RAM サイズを下表に示します。RL78/G2x シリーズから代表して RL78/G23 のコードサイズを掲載しています。

ROM (コードおよび定数) と RAM (グローバルデータ) のサイズは、ビルド時の「3 コンフィギュレーション」のコンフィギュレーションオプションによって決まります。

下表の値は下記条件で確認しています。

モジュールリビジョン: r_bsp v1.00

コンパイラバージョン: Renesas Electronics C/C++ Compiler Package for RL78 Family V1.09.00

LLVM C/C++ Compiler for Renesas RL78 7.0.0.202004

IAR C/C++ Compiler for Renesas RL78 version 4.20

コンフィグレーションオプション: デフォルト設定

ROM、RAM およびスタックのコードサイズ

デバイス	分類	使用メモリ					
		Renesas compiler		LLVM		IAR compiler	
		パラメータチェックあり	パラメータチェックなし	パラメータチェックあり	パラメータチェックなし	パラメータチェックあり	パラメータチェックなし
RL78/G23	ROM	1049	953	T.B.D	T.B.D	1400	1234
	RAM	0		T.B.D		0	

4.12　for 文、while 文、do while 文について

本モジュールでは、レジスタの反映待ち処理等で for 文、do while 文（ループ処理）を使用しています。これらループ処理には、「WAIT_LOOP」をキーワードとしたコメントを記述しています。そのため、ループ処理にユーザがフェイルセーフの処理を組み込む場合は、「WAIT_LOOP」で該当の処理を検索できます。

以下に記述例を示します。

```
for 文の例：
HIOSTOP = 0;
/* WAIT_LOOP */
for (w_count = 0U; w_count <= BSP_FIHWAITTIME; w_count++)
{
      BSP_NOP();
}

do while 文の例：
MSTOP = 0;
/* WAIT_LOOP */
do{
      tmp_stab_wait = OSTC;
      tmp_stab_wait &= STAB_WAIT;
}while(tmp_stab_wait != STAB_WAIT);
```

5. API 関数

5.1 概要

本モジュールでは、以下の関数を使用します。

関数	説明
R_BSP_StartClock	指定したクロックを発振する。
R_BSP_StopClock	指定したクロックの発振を停止する。
R_BSP_GetFclkFreqHz	CPU/周辺ハードウェア・クロックの周波数を返す。
R_BSP_SetClockSource	CPU/周辺ハードウェア・クロックのクロックソースを指定したクロックに変更する。
BSP_DI	すべてのマスカブル割り込みの受け付けを禁止します。本関数はマクロ関数です。
BSP_EI	すべてのマスカブル割り込みの受け付けを許可します。本関数はマクロ関数です。
BSP_NOP	NOP 命令を実行します。本関数はマクロ関数です。

5.2　R_BSP_StartClock()

この関数は、指定したクロックを発振します。

Format

```
bsp_err_t R_BSP_StartClock(clock_mode_t mode);
```

Parameters

mode

発振するクロックを指定(4.9.1 参照)

Return Values

BSP_OK　　/* 指定したクロックが正しく発振された */

BSP_ARG_ERROR　　/* 無効な引数が入力された */

Properties

r_bsp_common.h にプロトタイプ宣言されています。

Description

本関数は、指定したクロックを発振させます。

本関数を使用して高速システム・クロックまたはサブシステム・クロックを発振させる場合、クロック動作モード制御レジスタ(CMC)が正しく設定されている必要があります。

例えば、本関数の引数に高速システム・クロックを指定した場合でも、EXCLK/OSCSEL がポートに設定されていた場合、高速システム・クロックを発振させることができません。

CMC レジスタはリセット解除後、1 回のみ書き込み可能であるため、高速システム・クロック、サブシステム・クロックを使用する場合は初期設定時に有効に設定してください。

Example

```
bsp_err_t err;

/* Start High-speed on-chip oscillator */
err = R_BSP_StartClock(HIOCLK);

if (err != BSP_OK)
{
    /* NG processing */
}
```

Special Note:

なし

5.3　R_BSP_StopClock()

この関数は、指定したクロックの発振を停止します。ただし、CPU/周辺ハードウェア・クロックとして動作しているクロックを停止した場合の動作は保証しません。

Format

```
bsp_err_t R_BSP_StopClock(clock_mode_t mode);
```

Parameters

mode

発振を停止するクロックを指定(4.9.1 参照)

Return Values

BSP_OK　　/* 指定したクロックに対する停止処理が行われた */

BSP_ARG_ERROR　　/* 無効な引数が入力された */

Properties

r_bsp_common.h にプロトタイプ宣言されています。

Description

本関数は、指定したクロックの発振を停止させます。

本関数は指定するクロックに対するエラーチェックを行わないため、CPU/周辺ハードウェア・クロックとして使用しているクロックを停止した場合の動作を保証しません。

Example

```
bsp_err_t err;

/* Stop High-speed on-chip oscillator */
err = R_BSP_StopClock(HIOCLK);

if (err != BSP_OK)
{
    /* NG processing */
}
```

Special Note:

なし

5.4　R_BSP_SetClockSource()

この関数は、CPU/周辺ハードウェア・クロックに供給するクロックリソースを変更します。

高速システム・クロックまたはサブシステム・クロックに変更する場合、初期設定時に同クロックを有効にしておく必要があります。

同クロックの制御を行うクロック動作モード制御レジスタ(CMC)はリセット解除後 1 回のみ書き込み可能です。

そのため初期設定時に無効に設定した場合、動作中に有効にすることができません。

Format

```
bsp_err_t R_BSP_SetClockSource(clock_mode_t mode);
```

Parameters

mode

CPU/周辺ハードウェア・クロックに供給するクロックリソースを指定(4.9.1 参照)

Return Values

BSP_OK	/* CPU/周辺ハードウェア・クロックが指定したクロックに切り替えられた */
BSP_ERROR1	/* 指定されたクロックが発振していない */
BSP_ERROR2	/* CPU/周辺ハードウェア・クロックのリソース切り替え中に、発振していないクロックリソースに切り替えられる可能性がある状態移行が指定された */
BSP_ERROR3	/* 設定できない状態移行が指定された */
BSP_ARG_ERROR	/* 無効な引数が入力された */

Properties

r_bsp_common.h にプロトタイプ宣言されています。

Description

本関数は、CPU/周辺ハードウェア・クロックに供給するクロックリソースを変更します。

Example

```
bsp_err_t err;

/* Start clock operation(HIOCLK) */
err = R_BSP_StartClock(HIOCLK);

if(err != BSP_OK)
{
      /* NG processing */
}
/* Change clock source */
err = R_BSP_SetClockSource(HIOCLK);

if (err != BSP_OK)
{
      /* NG processing */
}
```

Special Note:

なし

5.5　R_BSP_GetFclkFreqHz()

この関数は、CPU/周辺ハードウェア・クロックの周波数を返します。

Format

```
uint32_t R_BSP_GetFclkFreqHz(void);
```

Parameters

なし

Return Values

CPU/周辺ハードウェア・クロックの周波数

Properties

r_bsp_common.h にプロトタイプ宣言されています。

Description

この関数は、CPU/周辺ハードウェア・クロックの周波数を返します。例えば、CPU/周辺ハードウェア・クロックが 20MHz になるように r_bsp_config.h を設定します。このとき、r_bsp によるクロック設定終了後、ユーザが CPU/周辺ハードウェア・クロックを 5MHz に変更した場合、関数の戻り値は’5000000’となります。

Example

```
uint32_t fclk_freq;

fclk_freq = R_BSP_GetFclkFreqHz();
```

Special Note:

なし

6. プロジェクトのセットアップ

ここでは r_bsp をプロジェクトに追加する方法を説明します。

6.1 FIT モジュールの追加方法

本モジュールは、使用するプロジェクトごとに追加する必要があります。ルネサスでは、Smart Configurator を使用した(1)、(3)の追加方法を推奨しています。ただし、Smart Configurator は、一部の RX デバイスのみサポートしています。サポートされていない RX デバイスについては(2)、(4)の方法を使用してください。

(1) e^2 studio 上で Smart Configurator を使用して FIT モジュールを追加する場合
e^2 studio の Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート(★RL78 用アプリケーションノートの名前と番号を確認して記述する)を参照してください。

(2) CS+上で Smart Configurator を使用して FIT モジュールを追加する場合
CS+上で、スタンドアロン版 Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート(★RL78 用アプリケーションノートの名前と番号を確認して記述する)を参照してください。

(3) IAREW 上で Smart Configurator を使用して FIT モジュールを追加する場合
スタンドアロン版 Smart Configurator を使用して、自動的にユーザプロジェクトに FIT モジュールを追加します。詳細は、アプリケーションノート(★RL78 用アプリケーションノートの名前と番号を確認して記述する)を参照してください。

6.2　e2studio プロジェクトへの FIT モジュールの追加方法

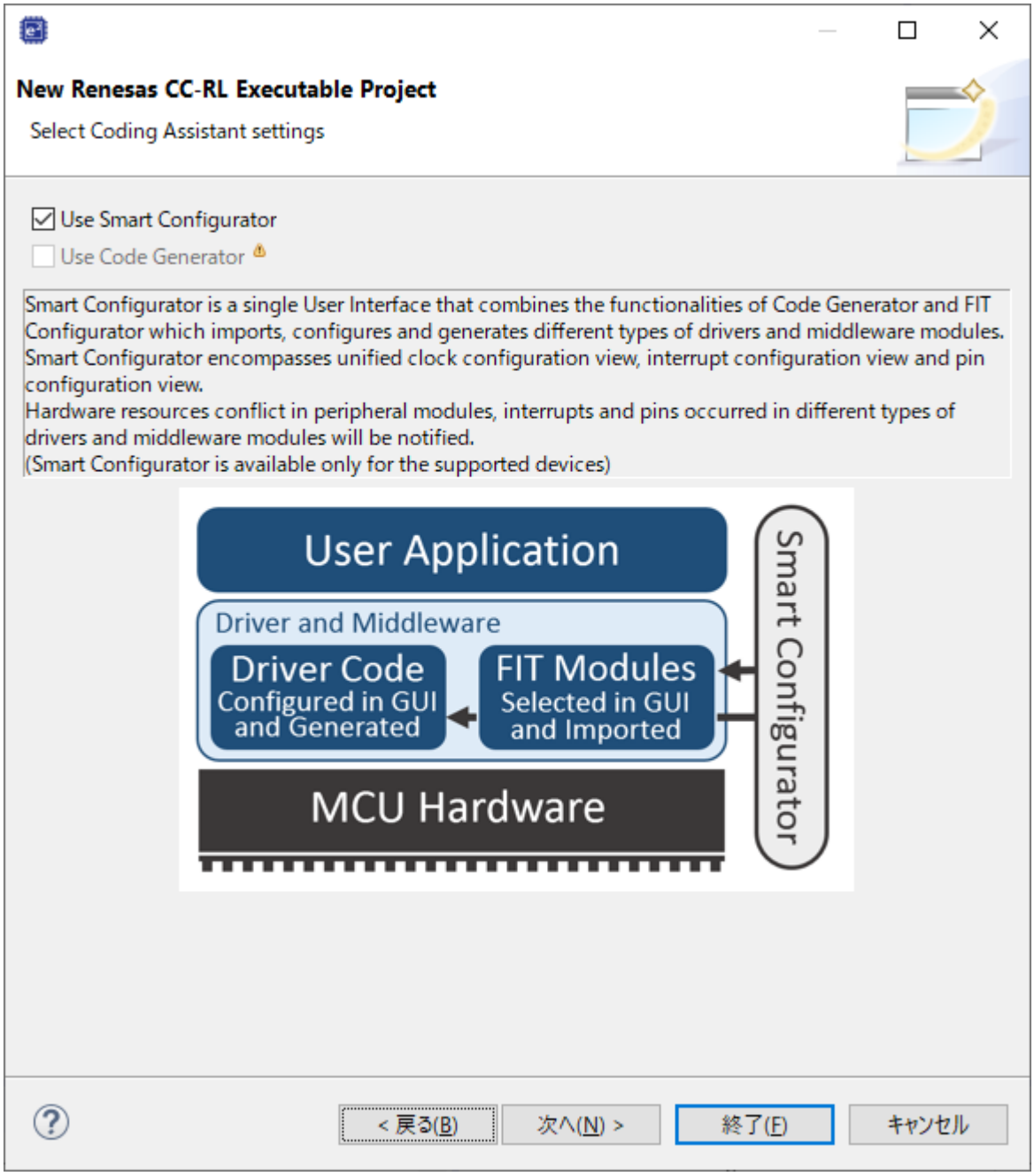
ここでは e2studio プロジェクトに FIT モジュールを追加する方法を説明します。

6.2.1　e2studio 上で Smart Configurator を使用して FIT モジュールを追加する方法

この説明では e2studio(★バージョン不明)を使用しています。

(1) e2studio で新規プロジェクトを作成します。

プロジェクト作成時、「Use Smart Configurator」のチェックを ON にすることで Smart Configurator を起動します。



(2) 「6.1 FIT モジュールの追加方法」の手順で e2studio プロジェクトに FIT モジュールを追加します。

(3) プロジェクト上で右クリックをして、”プロパティ”をクリックします。

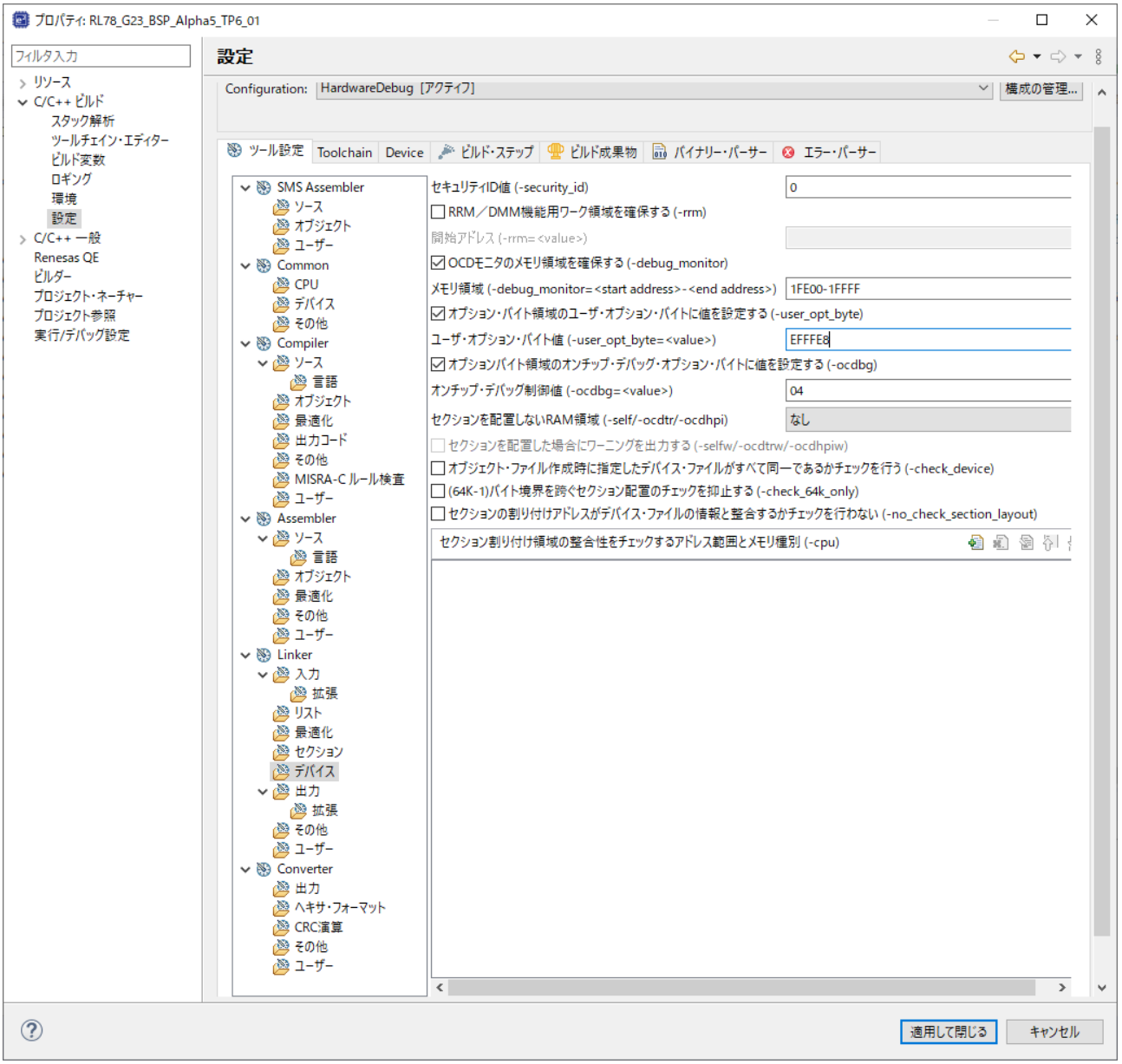
(4)　ツール設定タブの Compiler->ソースを選択します。

(5)　Smart Configulator で生成した FIT モジュールのインクルードパスが設定されています。



(6)　ツール設定タブの Linker->デバイスを選択します。

(7) オプション・バイト領域の設定を行う。



(8) プロジェクト上で右クリックをして、”プロジェクトのビルド”をクリックします。

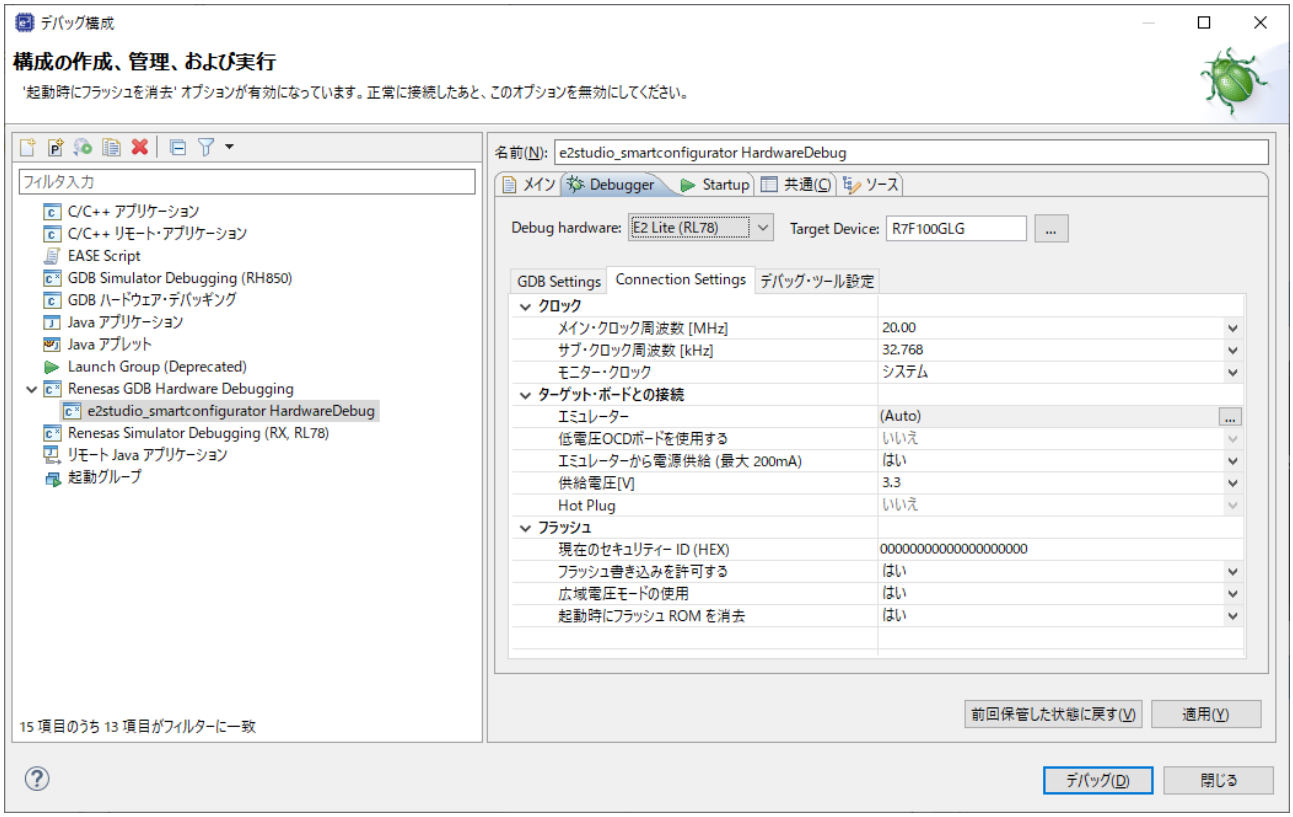
(9) プロジェクト上で右クリックして、デバッグ->デバッグの構成をクリックします。

(10) “Renesas GDB Hardware Debugging”->”プロジェクト名 HardwareDebug”をクリックします。

(11) Debugger タブで、Debug hardware に E2 Lite(RL78)を選択します。

(12) Connection Setting タグでメイン・クロック周波数とサブ・クロック周波数を設定します。

(13) Connection Setting タグでエミュレータからの電源供給を”はい”に設定します。



7.　付録

7.1　動作確認環境

本モジュールの動作確認環境を以下に示します。

表 7.1　動作確認環境 (Rev.1.00)

項目	内容
統合開発環境	ルネサスエレクトロニクス製 e^2 studio ★バージョン不明 IAR システムズ製 IAR Embedded Workbench for Renesas RL78 4.20.1
C コンパイラ	ルネサスエレクトロニクス製 C/C++ compiler for R78 family V.1.09.0 LLVM for Renesas RL78 Build Support 0.1.0.v20200629-1555
モジュールのリビジョン	Rev.1.00
使用ボード	★ボード名称不明 (型名：RTK7RLG230CLG000BJ)

改訂記録

Rev.	発行日	改訂内容	
		ページ	ポイント
1.00 Alpha5	Jul.31.20	—	初版発行 本ドキュメントは Alpha バージョンです。

製品ご使用上の注意事項

ここでは、マイコン製品全体に適用する「使用上の注意事項」について説明します。個別の使用上の注意事項については、本ドキュメントおよびテクニカルアップデートを参照してください。

1. 静電気対策
   CMOS 製品の取り扱いの際は静電気防止を心がけてください。CMOS 製品は強い静電気によってゲート絶縁破壊を生じることがあります。運搬や保存の際には、当社が出荷梱包に使用している導電性のトレーやマガジンケース、導電性の緩衝材、金属ケースなどを利用し、組み立て工程にはアースを施してください。プラスチック板上に放置したり、端子を触ったりしないでください。また、CMOS 製品を実装したボードについても同様の扱いをしてください。
2. 電源投入時の処置
   電源投入時は、製品の状態は不定です。電源投入時には、LSI の内部回路の状態は不確定であり、レジスタの設定や各端子の状態は不定です。外部リセット端子でリセットする製品の場合、電源投入からリセットが有効になるまでの期間、端子の状態は保証できません。同様に、内蔵パワーオンリセット機能を使用してリセットする製品の場合、電源投入からリセットのかかる一定電圧に達するまでの期間、端子の状態は保証できません。
3. 電源オフ時における入力信号
   当該製品の電源がオフ状態のときに、入力信号や入出力プルアップ電源を入れないでください。入力信号や入出力プルアップ電源からの電流注入により、誤動作を引き起こしたり、異常電流が流れ内部素子を劣化させたりする場合があります。資料中に「電源オフ時における入力信号」についての記載のある製品は、その内容を守ってください。
4. 未使用端子の処理
   未使用端子は、「未使用端子の処理」に従って処理してください。CMOS 製品の入力端子のインピーダンスは、一般に、ハイインピーダンスとなっています。未使用端子を開放状態で動作させると、誘導現象により、LSI 周辺のノイズが印加され、LSI 内部で貫通電流が流れたり、入力信号と認識されて誤動作を起こす恐れがあります。
5. クロックについて
   リセット時は、クロックが安定した後、リセットを解除してください。プログラム実行中のクロック切り替え時は、切り替え先クロックが安定した後に切り替えてください。リセット時、外部発振子（または外部発振回路）を用いたクロックで動作を開始するシステムでは、クロックが十分安定した後、リセットを解除してください。また、プログラムの途中で外部発振子（または外部発振回路）を用いたクロックに切り替える場合は、切り替え先のクロックが十分安定してから切り替えてください。
6. 入力端子の印加波形
   入力ノイズや反射波による波形歪みは誤動作の原因になりますので注意してください。CMOS 製品の入力がノイズなどに起因して、V_{IL}（Max.）から V_{IH}（Min.）までの領域にとどまるような場合は、誤動作を引き起こす恐れがあります。入力レベルが固定の場合はもちろん、V_{IL}（Max.）から V_{IH}（Min.）までの領域を通過する遷移期間中にチャタリングノイズなどが入らないように使用してください。
7. リザーブアドレス（予約領域）のアクセス禁止
   リザーブアドレス（予約領域）のアクセスを禁止します。アドレス領域には、将来の拡張機能用に割り付けられている リザーブアドレス（予約領域）があります。これらのアドレスをアクセスしたときの動作については、保証できませんので、アクセスしないようにしてください。
8. 製品間の相違について
   型名の異なる製品に変更する場合は、製品型名ごとにシステム評価試験を実施してください。同じグループのマイコンでも型名が違うと、フラッシュメモリ、レイアウトパターンの相違などにより、電気的特性の範囲で、特性値、動作マージン、ノイズ耐量、ノイズ幅射量などが異なる場合があります。型名が違う製品に変更する場合は、個々の製品ごとにシステム評価試験を実施してください。

ご注意書き

1. 本資料に記載された回路、ソフトウェアおよびこれらに関連する情報は、半導体製品の動作例、応用例を説明するものです。お客様の機器・システムの設計において、回路、ソフトウェアおよびこれらに関連する情報を使用する場合には、お客様の責任において行ってください。これらの使用に起因して生じた損害（お客様または第三者いずれに生じた損害も含みます。以下同じです。）に関し、当社は、一切その責任を負いません。
2. 当社製品、本資料に記載された製品データ、図、表、プログラム、アルゴリズム、応用回路例等の情報の使用に起因して発生した第三者の特許権、著作権その他の知的財産権に対する侵害またはこれらに関する紛争について、当社は、何らの保証を行うものではなく、また責任を負うものではありません。
3. 当社は、本資料に基づき当社または第三者の特許権、著作権その他の知的財産権を何ら許諾するものではありません。
4. 当社製品を、全部または一部を問わず、改造、改変、複製、リバースエンジニアリング、その他、不適切に使用しないでください。かかる改造、改変、複製、リバースエンジニアリング等により生じた損害に関し、当社は、一切その責任を負いません。
5. 当社は、当社製品の品質水準を「標準水準」および「高品質水準」に分類しており、各品質水準は、以下に示す用途に製品が使用されることを意図しております。
   標準水準：　コンピュータ、OA 機器、通信機器、計測機器、AV 機器、家電、工作機械、パーソナル機器、産業用ロボット等
   高品質水準：輸送機器（自動車、電車、船舶等）、交通制御（信号）、大規模通信機器、金融端末基幹システム、各種安全制御装置等
   当社製品は、データシート等により高信頼性、Harsh environment 向け製品と定義しているものを除き、直接生命・身体に危害を及ぼす可能性のある機器・システム（生命維持装置、人体に埋め込み使用するもの等）、もしくは多大な物的損害を発生させるおそれのある機器・システム（宇宙機器と、海底中継器、原子力制御システム、航空機制御システム、プラント基幹システム、軍事機器等）に使用されることを意図しておらず、これらの用途に使用することは想定していません。たとえ、当社が想定していない用途に当社製品を使用したことにより損害が生じても、当社は一切その責任を負いません。
6. 当社製品をご使用の際は、最新の製品情報（データシート、ユーザーズマニュアル、アプリケーションノート、信頼性ハンドブックに記載の「半導体デバイスの使用上の一般的な注意事項」等）をご確認の上、当社が指定する最大定格、動作電源電圧範囲、放熱特性、実装条件その他指定条件の範囲内でご使用ください。指定条件の範囲を超えて当社製品をご使用された場合の故障、誤動作の不具合および事故につきましては、当社は、一切その責任を負いません。
7. 当社は、当社製品の品質および信頼性の向上に努めていますが、半導体製品はある確率で故障が発生したり、使用条件によっては誤動作したりする場合があります。また、当社製品は、データシート等において高信頼性、Harsh environment 向け製品と定義しているものを除き、耐放射線設計を行っておりません。仮に当社製品の故障または誤動作が生じた場合であっても、人身事故、火災事故その他社会的損害等を生じさせないよう、お客様の責任において、冗長設計、延焼対策設計、誤動作防止設計等の安全設計およびエージング処理等、お客様の機器・システムとしての出荷保証を行ってください。特に、マイコンソフトウェアは、単独での検証は困難なため、お客様の機器・システムとしての安全検証をお客様の責任で行ってください。
8. 当社製品の環境適合性等の詳細につきましては、製品個別に必ず当社営業窓口までお問合せください。ご使用に際しては、特定の物質の含有・使用を規制する RoHS 指令等、適用される環境関連法令を十分調査のうえ、かかる法令に適合するようご使用ください。かかる法令を遵守しないことにより生じた損害に関して、当社は、一切その責任を負いません。
9. 当社製品および技術を国内外の法令および規則により製造・使用・販売を禁止されている機器・システムに使用することはできません。当社製品および技術を輸出、販売または移転等する場合は、「外国為替及び外国貿易法」その他日本国および適用される外国の輸出管理関連法規を遵守し、それらの定めるところに従い必要な手続きを行ってください。
10. お客様が当社製品を第三者に転売等される場合には、事前に当該第三者に対して、本ご注意書き記載の諸条件を通知する責任を負うものといたします。
11. 本資料の全部または一部を当社の文書による事前の承諾を得ることなく転載または複製することを禁じます。
12. 本資料に記載されている内容または当社製品についてご不明な点がございましたら、当社の営業担当者までお問合せください。

注 1. 本資料において使用されている「当社」とは、ルネサス エレクトロニクス株式会社およびルネサス エレクトロニクス株式会社が直接的、間接的に支配する会社をいいます。

注 2. 本資料において使用されている「当社製品」とは、注 1 において定義された当社の開発、製造製品をいいます。

(Rev.4.0-1　2017.11)

本社所在地

〒135-0061 東京都江東区豊洲 3-2-24（豊洲フォレシア）

www.renesas.com

お問合せ窓口

弊社の製品や技術、ドキュメントの最新情報、最寄の営業お問合せ窓口に関する情報などは、弊社ウェブサイトをご覧ください。

www.renesas.com/contact/

商標について

ルネサスおよびルネサスロゴはルネサス エレクトロニクス株式会社の商標です。すべての商標および登録商標は、それぞれの所有者に帰属します。

© 2020 Renesas Electronics Corporation. All rights reserved.